

Design Book API — UI Element Map

Every control in the LEICHT catalog browser, named as the user sees it and wired to the exact `design-book` endpoint and field that powers it. Mock-ups reproduce the live Instrument 1:1.

Backend NestJS · /design-book

Endpoints 11

SKUs 16,393

Cards 1,338

Programmes 120

HOW TO READ THIS

The wiring convention

Each section pairs a re-creation of one UI region with a table. Numbered markers (1 ...) on the mock-up point to the matching row. A row names the **visible element**, the endpoint that serves it, and the request param or response field involved.

GET	Read endpoint — lists, lookups, detail, config resolution.
POST	Write endpoint — catalog ingest / sync only.
?param	Request query/path parameter the control sets.
field.path	Response field the control renders.

Masthead — collection & live counts

The header line "16,384 units · 120 programmes" is the catalog's heartbeat: it comes straight off the ingest run, not a client-side count.

LIVE UI

1

ICHT

PRIMO · DESIGNER INSTRUMENT

Collection 2026/2 · IDM 3.0.1

16,384 units · 120 programmes

2

ELEMENT	ENDPOINT	FIELD
1 Collection line	GET /meta	meta.collection, catalogVersion
2 Units · programmes count	GET /stats	activeSkus, programs

Programme selector, Mix & ANTOSO

Choosing a kitchen line re-prices and re-gates the whole catalog. The dropdown is fed by the programme table; Mix and ANTOSO read the rule tables shipped in `meta.ruleTables`.

LIVE UI

1

No programme · point range

2

Mix

3

ANTOSO floating

ELEMENT	ENDPOINT	PARAM / FIELD
1 Programme dropdown	GET /programs	lists key · name · family · priceField; selection feeds <code>?programKey</code> on lists & prices detail
2 Mix (per-area line)	GET /meta	ruleTables.programs_special + <code>?programKey</code> per area
3 ANTOSO floating / wall	GET /meta	ruleTables.antoso

Width · Height · Depth filters

The dimension chips are pure query parameters — they narrow both the card grid and the SKU list. Values are in centimetres, matching the stored `widthCm / heightCm / depthCm` .

LIVE UI

W

1

All

15

20

30

45

60

80

100

120

Corner

H

2

All

73

80

86

☐ GREY, DON'T HIDE

4

D

36

48

58

68

63

3

ELEMENT	ENDPOINT	PARAM
1 Width chips	GET /cards · GET /items	?widthCm=60 (All = omit)
2 Height chips	GET /cards · GET /items	?heightCm=80
3 Depth chips (carcass)	GET /items · GET /resolve	?depthCm=58 → depth-encoded code
4 Grey, don't hide	Client render flag — show off-programme units greyed instead of filtered. Pair with ?active=false to include inactive SKUs.	

Category tree & counts

The navigation tree and every count next to it come from one call. Selecting a row sets the category / subcategory filter on the grid.

LIVE UI

CATEGORY	
All categories	1632
Base	218
All Base	218
Sinks	24
Cooktops & Downdrafts	31
Drawers & Pullouts	30
Tall	277
Wall	68

ELEMENT	ENDPOINT	PARAM / FIELD
1 Category row + count	GET /categories	category, productCount; click sets ?category=Base
2 Subcategory / section	GET /categories	subcategories[].sections[]; click sets ?subcategory · ?section

Count note. The live rail shows **1632** cards (Base 218); the client v584 export we ingest holds **1338** cards (Base 170). Per the locked decision the **export is source of truth** — the deployed UI is a reference build and may split cards differently. /categories reflects the export.

Product card (grid view)

Each grid tile is one configurable card: SKUs grouped by `cardId` . The header label + programme badge, the W/H/D chips and the tier dots all come from `/cards`; clicking any chip resolves to a real order code via `/resolve`.

LIVE UI · CARD

Cooktop Unit

ALL

Cooktop Unit · Top Blender · BZ2

TK6080BZ2

W 600 · H 795 mm

H

80

W

60

70

80

90

100

D

58

63

68

P

P1

C

C1

A

ELEMENT	ENDPOINT	FIELD / PARAM
1 Group label	GET /cards	cardName, section
2 Programme badge	GET /cards	programmeBadge — ALL / P · C / C (from tier families)
3 W / H / D chips	GET /resolve	chip set = options.widthsCm / heightsCm / depthsCm; click → ?cardId&widthCm=... → order code
4 Tier dots P·P1·C·C1·A	GET /resolve	base tiers = tierCodes; premium P1/C1 = skuData.featureFlags → code P1<sku>
♥ list · 📄 copy		Client-local (My-list / clipboard). No backend call today — candidate favorites endpoint.

The grid itself ("209 types") is `GET /cards` with the active category/size/programme filters and `?page` / `?limit` ; the count is the response `pagination.total`.

Item detail panel

Clicking a card opens the detail slide-over. One call — `GET /detail/:sku` — returns { item, detail }; every block below is a key of the computed `detail` object, built server-side to mirror the legacy panel exactly.

LIVE UI · PANEL

BASE · COOKTOPS & DOWNDRAFTS 1

Cooktop Unit · Top Blender · BZ2

TK6080BZ2

Copy

Catalog

CONFIGURE 2

WIDTH

60708090100120

HEIGHT

H73H80H86

DEPTH

586368

PROGRAMME

PP1CC1A

DESCRIPTION 3

Hob unit

1 blender

2 pullouts

SPECIFICATION 4

Width

600 mm

Height

795 mm

Depth

560 mm carcass

Carcase line

80

Weight

46.4 kg

Volume	0.29 m ³
Catalog page	22.09 · PDF p.267
PROGRAMME AVAILABILITY	5
Available in all programmes.	
RESTRICTIONS	6
- No recessed handle vertical - Cupboards 45° chamfered edge on both sides - Fronts –Q handle 790, 791	
MODIFICATIONS – HOW TO	7
P1 — Primo, one handle on top — order <code>P1TK6080BZ2</code> . Bottom drawers open 760 (touch) or 761 (electric).	
Handle-less mechanical (760) — Softclose touch latch. <code>set handle 760</code>	
Handle-less electric (761) — SensoMatic. <code>set handle 761</code> <code>+SMNT</code> 761 electric ALWAYS needs the SMNT transformer — not added automatically	
ALTERATIONS & ACCESSORIES	8
Overview Installation 1 Visible Sides 3 Alterations 22 Pullouts 4	
PLANNING NOTES	9
Control panel on top.	

PANEL BLOCK	DETAIL.FIELD
1 Breadcrumb + title	header.category · subcategory · cardName · code
2 Configure pills	configure.{width,height,depth,programme}.options[].code — each pre-resolved to its order code (same as <code>GET</code> /resolve)

3	Description	description[]
4	Specification rows	specification[] — wt/10 kg, vl/1000 m³, priceGroupRef · pp
5	Programme availability	programmeAvailability — restricted + inclusion programs []
6	Restrictions	restrictions[] (technicalNotes)
7	Modifications — how to	modifications[] — P1/C1 premium, 760/761 handle-less, vertical-V; each with codes[] + warn
8	Alterations tabs	alterations[].{name,count,codes} (relatedSkus + accessories)
9	Planning notes	planningNotes[]

Also on the payload, not shown in this panel: detail.pricing (priceClass · priceMatrix · bookPriceByProgram) and detail.clickNote. Book price for programme P = priceMatrix[programs[P].priceField]. The Instrument surfaces price through the separate **Pricing** admin, not the detail slide-over.

All 11 endpoints

Complete surface of the `/design-book` module. Every route is JWT-guarded and documented with an `@ApiOperation` in Swagger.

ENDPOINT	PURPOSE	POWERS UI
POST <code>/ingest</code>	Upload full catalog export; upsert every SKU + reference table; missing → inactive.	Admin sync (CSV↑ / data refresh)
GET <code>/cards</code>	Card-level grid list; SKUs grouped by cardId with axes + programme badge.	Result grid · counts
GET <code>/cards/:cardId</code>	Every SKU of one card (incl. inactive), sorted.	Card expand / variant set
GET <code>/items</code>	Paginated SKU list/search; category/size/programme filters + free-text q.	Search · size filters
GET <code>/items/:sku</code>	Look up a code (array — some codes cross-listed across cards).	Deep-link / code lookup
GET <code>/detail/:sku</code>	Raw item + computed detail (configure/spec/pricing/mods/...).	Detail slide-over
GET <code>/resolve</code>	cardId + axis values → target order code.	Every W/H/D/programme pill click
GET <code>/programs</code>	All programmes (key·name·family·priceField).	Programme dropdown · Mix
GET <code>/categories</code>	Category → subcategory → section tree with counts.	Left rail
GET <code>/meta</code>	Singleton reference: meta, priceFieldLegend, ruleTables, last ingest.	Pricing · ANTOSO · handle recipes
GET <code>/stats</code>	Total / active / inactive SKUs, cards, programmes, catalog version.	Masthead counts